HK Software guide

Alberto Montanelli

May 7, 2026

Contents

1 Containerization

1.1 What is a container?

A container is a self-contained software environment that includes an application together with the libraries, tools and runtime dependencies needed to execute it. The main idea is to make the software reproducible and portable: instead of installing all packages manually on every machine, one builds a container image once and then runs it wherever a compatible container runtime is available.

A **container image** can be thought of as a frozen snapshot of a working software environment. For example `fiTQun_latest.sif` is Apptainer image containing a predefined software setup.

1.2 Docker and Apptainer

Two common container tools are Docker and Apptainer.

Docker. Docker is widely used to build, distribute and run container images. Images are usually written through a `Dockerfile` and can be downloaded from registries such as Docker Hub or GitHub Container Registry.
For example:

```
docker pull wcsim/wcsim:latest
```

downloads a Docker image from Docker Hub.

Apptainer. Apptainer, formerly Singularity, is commonly used in HPC environments. It is designed to run containers without requiring root privileges from the user. It can run local `.sif` images and can also pull images from Docker-compatible registries.
For example:

```
apptainer pull wcsim_latest.sif docker://wcsim/wcsim:latest
```

downloads a Docker image and converts it into an Apptainer `.sif` image.

1.3 Where container images can be stored

A container image is just a file. For Apptainer, the usual image format is `.sif`. This means that the image can be stored:

- on the local machine;
- on a remote server;
- in a shared filesystem accessible from several machines;
- in a project directory on a cluster.

If the image is stored on a shared filesystem, different users or different compute nodes can run the same software environment without reinstalling it.

1.4 Basic Apptainer commands

1.4.1 Check that Apptainer is available

```
apptainer --version
```

1.4.2 Run a command inside a container

The most common command is `apptainer exec`. It runs a command inside the container and then exits.

```
apptainer exec image.sif command
```

Example:

```
apptainer exec fiTQun_latest.sif which root
```

With an environment setup script:

```
apptainer exec fiTQun_latest.sif bash ./fiTQun_env.sh which runfiTQun
```

In this example, `fiTQun_env.sh` sets the paths to fiTQun, ROOT and WCSimRoot before executing the requested command.

1.4.3 Open an interactive shell

To enter the container interactively:

```
apptainer shell image.sif
```

Inside the shell, one can run commands as if working inside the container:

```
which root
which runfiTQun
which WCSim
```

To exit:

```
exit
```

However, one important point must be kept in mind: Apptainer usually binds the current working directory and the user home directory from the host system into the container. Therefore, if the container is opened from a directory on the host machine, running `ls` show the content of the current host directory, not the internal software directories of the container. Inside the shell, one can run `ls /` to look at the actual content of the container. In the following a demonstrative example is shown:

```
[cc@marduk HyperKamiokande]$ ls
fiTQun_env.sh fiTQun_latest.sif fiTQun_WCSim.sif
fiTQun_git_repo fiTQun_WCSim.def utils
[cc@marduk HyperKamiokande]$ apptainer shell fiTQun_WCSim.sif
Apptainer> ls
fiTQun_env.sh fiTQun_latest.sif fiTQun_WCSim.sif
fiTQun_git_repo fiTQun_WCSim.def utils
Apptainer> ls /
bin dev etc lib lost+found mnt proc run singularity sys usr
boot environment home lib64 media opt root sbin srv tmp var
Apptainer> cd usr
bash: cd: usr: No such file or directory
Apptainer> cd /usr
```

1.4.4 Pull an image from a registry

Apptainer can pull images from Docker-compatible registries and convert them to `.sif` format.
From Docker Hub:

```
apptainer pull wcsim_latest.sif docker://wcsim/wcsim:latest
```

From GitHub Container Registry:

```
apptainer pull hk_software.sif docker://ghcr.io/hyperk/hk-software:0.0.2
```

In the case of fiTQun:

```
apptainer pull namefile.sif docker://registry.git.hyperk.org/hyperk/recon/fitqun:latest
```

1.4.5 Build an image from a definition file

An Apptainer image can be built from a definition file:

```
apptainer build output_image.sif recipe.def
```

Example:

```
apptainer build fiTQun_WCSim.sif fiTQun_WCSim.def
```

The definition file describes the base image, the installation commands and the environment variables that should be available inside the final image.
On some systems, building an image may require root privileges because the build process has to install packages and write into system directories inside the container image. If the system supports user namespaces, Apptainer can emulate this with:

```
apptainer build --fakeroot output_image.sif recipe.def
```

The option `-fakeroot` does not make the user root on the host machine, it only gives the build process root-like privileges inside the isolated build environment. This is useful, for example, when the definition file contains commands such as:

```
yum install ...
apt-get install ...
mkdir /usr/local/hk/...
```

Without `-fakeroot`, these operations may fail if the user does not have the required permissions. Whether `-fakeroot` works depends on the configuration of the server or cluster.

1.4.6 Inspect an image

The command `apptainer inspect` is used to read metadata stored inside an image without opening an interactive shell.
The general command is:

```
apptainer inspect image.sif
```

This gives general information about the image. More specific options are often more useful.
To inspect the environment variables defined by the image:

```
apptainer inspect --environment image.sif
```

For example, in the image used here this command shows variables such as:

```
export FITQUN_SRC=/usr/local/hk/fiTQun
export FITQUN_INSTALL=/usr/local/hk/fiTQun/install-Linux_x86_64-gcc_8-python_3.12.1
export ROOT_INSTALL=/usr/local/hk/ROOT/install-Linux_x86_64-gcc_8-python_3.12.1
export GEANT4_INSTALL=/usr/local/hk/Geant4/install-10.3.3
export WCSIM_HOME=/usr/local/hk/WCSimFull
export WCSIM_INSTALL_DIR=/usr/local/hk/WCSimFull/install
```

This is useful to understand which software paths were stored in the image at build time.
To inspect the runscript:

```
apptainer inspect --runscript image.sif
```

For example, the runscript of the test image prints:

```
echo "fiTQun + full WCSim test image"
echo "Use:"
echo " bash /usr/local/hk/fitqun_wcsim_env.sh which WCSim"
echo " bash /usr/local/hk/fitqun_wcsim_env.sh which runfiTQun"
```

This means that running:

```
apptainer run image.sif
```

will not start WCSim or fiTQun directly. It will only print these instructions.

1.4.7 Bind host directories inside the container

A container image contains the software environment, but input and output data are often stored outside the image. The `-bind` option is used to mount a directory from the host system inside the container.
The general syntax is:

```
apptainer exec --bind /host/path:/container/path image.sif command
```

Here:

- `/host/path` is the directory on the real machine;
- `/container/path` is the path where that directory will appear inside the container.

For example:

```
apptainer exec \
    --bind /data/hk:/data/hk \
    fiTQun_WCSim.sif \
    bash /usr/local/hk/fiTQun_WCSim_env.sh \
    ls /data/hk
```

This makes the host directory `/data/hk` visible inside the container as `/data/hk`.
This is useful, for example, when WCSim output ROOT files are stored outside the container and one wants to process them with fiTQun inside the container:

```
apptainer exec \
    --bind /data/hk:/data/hk \
    fiTQun_WCSim.sif \
    bash /usr/local/hk/fiTQun_WCSim_env.sh \
    runfiTQun \
        -n 1 \
        -r /data/hk/fitqun_output.root \
        -p /path/to/HyperK.parameters.dat \
        /data/hk/wcsim_input.root
```

In this example:

- the file `wcsim_input.root` is physically stored on the host machine in `/data/hk`;
- the container sees it as `/data/hk/wcsim_input.root`;
- fiTQun runs inside the container;
- the output file `fitqun_output.root` is written back to the host directory.

One can also bind a directory to a different path inside the container:

```
apptainer exec \
    --bind /scratch/cc/hk_data:/data \
    fiTQun_WCSim.sif \
    bash /usr/local/hk/fiTQun_WCSim_env.sh \
    ls /data
```

Here, the host directory `/scratch/cc/hk_data` appears inside the container as `/data`.

1.5 Practical remarks

Container images should be treated as versioned software environments. If WCSim, fiTQun, ROOT or Geant4 are updated, a new image should be built and saved with a clear name, for example:

```
fitqun_plus_wcsim_test_v1.sif
fitqun_plus_wcsim_test_v2.sif
```

For reproducibility, it is useful to store a small text file inside or next to the image containing the relevant software versions:

```
ROOT version
Geant4 version
WCSim commit
fiTQun commit
WCSimRoot version
build date
```

This avoids ambiguity when results are produced at different times or on different machines.

2 WCSim

2.1 Purpose of WCSim

WCSim is a Geant4-based simulation program for water Cherenkov detectors. It is used to simulate the detector geometry, particle propagation, Cherenkov light production, PMT response and detector-level output. Its main dependencies are `ROOT` and `Geant4`. **The output is stored in ROOT files using the WCSimRoot classes**, which can then be read by reconstruction software such as fiTQun.

In the context of this work, WCSim is needed for event simulation, while fiTQun is used for event reconstruction. This distinction is important: an installation containing only WCSimRoot is sufficient for reading WCSim output files, but it is not sufficient to generate new simulated events.

2.2 WCSim as a Docker image

The Docker Hub repository `wcsim/wcsim` provides pre-built Docker images of WCSim. In principle, one can pull the latest image with:

```
docker pull wcsim/wcsim:latest
```

or convert/pull it directly with `apptainer`:

```
apptainer pull wcsim_latest.sif docker://wcsim/wcsim:latest
```

However, these images do not appear to be actively maintained since they were last pushed approximately five years ago. For this reason, this option isn't ideal for the present setup, where a more recent WCSim version is needed for a more probable compatibility with the existing fiTQun container.

2.3 WCSim Dockerfile

The WCSim GitHub repository also contains a `Dockerfile`. This file starts from:

```
FROM wcsim/wcsim:base
```

and then clones the WCSim repository, creates separate build and install directories, runs CMake, and installs WCSim:

```
RUN git clone https://github.com/WCSim/WCSim
RUN mkdir $HYPERKDIR/WCSim-build $HYPERKDIR/WCSim-install

WORKDIR $HYPERKDIR/WCSim-build

RUN source $HYPERKDIR/env-WCSim.sh &&\
    cmake3 $HYPERKDIR/WCSim &&\
    make -j‘nproc‘ install
```

This means that the Dockerfile provides a recipe to build WCSim inside a base environment that already contains the required dependencies. The base image is therefore the important part: it must provide a working compiler, ROOT, Geant4 and the Geant4 data files.

2.4 WCSim CI workflows

The WCSim repository uses GitHub Actions for build and physics validation tests.
The workflow `build_and_physics_test.yml` runs inside the container:

`docker://ghcr.io/hyperk/hk-software:0.0.2`

which provides the software prerequisites used by the automated tests. The workflow then checks out the WCSim source code, checks out the separate WCSim Validation repository, sets up the environment, installs additional packages such as HepMC3, builds WCSim, and runs the validation tests. In practice, the CI uses a prepared base image containing the required dependencies and then builds the current WCSim source code inside it.
This base image (`docker://ghcr.io/hyperk/hk-software:0.0.2`) is therefore the most relevant reference environment for WCSim, since it is the one used by the current WCSim automated build and validation workflow.

3 fiTQun

3.1 Purpose of fiTQun

fiTQun is a likelihood-based reconstruction software package for water Cherenkov detectors. It is used to reconstruct event-level quantities such as the interaction vertex, particle direction, momentum, number of rings and particle identification. In the context of this work, fiTQun is used as the event reconstruction software, while WCSim is used as the detector simulation software.

This distinction is important. WCSim simulates the detector response starting from particle propagation and Cherenkov light production, while fiTQun takes as input already simulated detector-level information, such as PMT hit times and charges, and performs the reconstruction. Therefore, fiTQun does not need the full Geant4 simulation machinery at runtime if the input ROOT files have already been produced by WCSim. It only needs the classes required to read the WCSim ROOT output, namely the WCSimRoot interface.

3.2 fiTQun Docker image

The starting point of this work was the pre-existing Apptainer image:

```
fiTQun_latest.sif
```

which was obtained from the `latest` container image produced by the fiTQun GitHub deployment pipeline.

The fiTQun repository contains a single Dockerfile:

```
./Dockerfile
```

The Dockerfile starts from the Hyper-K installation framework image:

```
FROM registry.git.hyperk.org/hyperk/hk-pilot:latest
```

This means that the fiTQun image is not built from a generic Linux base image. Instead, it is built on top of `hk-pilot`, the Hyper-K software installation framework.
The Dockerfile then copies a few already-built external packages from the `hk-meta-externals:latest` image:

```
COPY --from=registry.git.hyperk.org/hyperk/externals/hk-meta-externals:latest \
    /usr/local/hk/ROOT /usr/local/hk/ROOT

COPY --from=registry.git.hyperk.org/hyperk/externals/hk-meta-externals:latest \
    /usr/local/hk/WCSimRoot /usr/local/hk/WCSimRoot

COPY --from=registry.git.hyperk.org/hyperk/externals/hk-meta-externals:latest \
    /usr/local/hk/ToolFrameworkCore /usr/local/hk/ToolFrameworkCore
```

Therefore, the image does not build `ROOT`, `WCSimRoot` or `ToolFrameworkCore` from source during the fiTQun image creation. These packages are inherited from a separate pre-built external image.
The fiTQun repository itself is then copied into the image:

```
COPY . /usr/local/hk/fiTQun
```

and fiTQun is installed through `hk-pilot`:

```
WORKDIR /usr/local/hk
RUN . /usr/local/hk/hk-pilot/setup.sh &&\
    hkp install -r --exclude-externals fiTQun
```

The option `-exclude-externals` is important: it means that the external dependencies are not rebuilt by this command. Instead, the build uses the already available packages copied from `hk-meta-externals:latest`.

Thus, **the production chain of the `fiTQun:latest` image** can be summarized as:

```
GitHub workflow
  -> Dockerfile
    -> FROM hk-pilot:latest
    -> copy ROOT from hk-meta-externals:latest
    -> copy WCSimRoot from hk-meta-externals:latest
    -> copy ToolFrameworkCore from hk-meta-externals:latest
    -> copy fiTQun source code
    -> hkp install -r --exclude-externals fiTQun
```

3.3 fiTQun installation through hk-pilot

The fiTQun repository contains an `hkinstall.py` file which defines how `hk-pilot` should install the package. The relevant part is:

```
class fiTQun(CMake):

    def __init__(self, path):
        super().__init__(path)

        self._package_name = "fiTQun"
        self._cmakelist_path = "."
        self._cmake_options = {
            "CMAKE_CXX_STANDARD": "14",
            "HEMI_CUDA_DISABLE": "ONE",
            "fiTQun_PerfTracker_ENABLED": "OFF",
            "CMAKE_BUILD_TYPE": "Release"
        }
```

This shows that fiTQun is installed as a CMake package through the generic `CMake` class provided by `hk-pilot`. The build is performed in release mode, using C++14, and with the performance tracker disabled.

The custom post-install step only appends the `FITQUN_ROOT` variable to the generated setup file:

```
def post_install(self):
    super().post_install()
    with open(os.path.join(self._install_folder, "setup.sh"), "a") as file:
        file.write(f"# FITQUN_ROOT variable\n")
        file.write(f"export FITQUN_ROOT={self.path}\n")
    return True
```

Therefore, the actual dependency resolution is not mainly encoded in `hkinstall.py`. The important dependency information is instead found in the CMake configuration files and in the external packages copied by the Dockerfile. In particular, the fiTQun CMake configuration searches for `WCSimRoot`:

```
find_package(WCSimRoot)
```

and the build defines:

```
NOSKLIBRARIES
```

This indicates that the image is built in the mode which does not rely on the Super-K libraries, but instead uses the WCSimRoot interface to read WCSim ROOT files.

3.4 Contents of the fiTQun latest image

The available executables inside the image were checked, showing that `runfiTQun` was available:

```
/usr/local/hk/fiTQun/install-Linux_x86_64-gcc_8-python_3.12.1/bin/runfiTQun
```

while `runfiTQunWC` and `WCSim` were not found.
The image also contained setup scripts for the following packages:

```
/usr/local/hk/Catch2/install-Linux_x86_64-gcc_8-python_3.12.1/setup.sh
/usr/local/hk/fiTQun/install-Linux_x86_64-gcc_8-python_3.12.1/setup.sh
/usr/local/hk/hk-DataModel/install-Linux_x86_64-gcc_8-python_3.12.1/setup.sh
/usr/local/hk/hk-pilot/setup.sh
/usr/local/hk/ROOT/install-Linux_x86_64-gcc_8-python_3.12.1/setup.sh
/usr/local/hk/ToolFrameworkCore/install-Linux_x86_64-gcc_8-python_3.12.1/setup.sh
/usr/local/hk/WCSimRoot/install-Linux_x86_64-gcc_8-python_3.12.1/setup.sh
```

Thus, the relevant packages present in the image are:

- `fiTQun`;
- `ROOT`;
- `WCSimRoot`;
- `ToolFrameworkCore`;
- `hk-DataModel`;
- `Catch2`;
- `hk-pilot`.

On the other hand, the image does not contain:

- the full WCSim executable;
- the full WCSim simulation library;
- `Geant4`;
- the Geant4 data files.

3.5 WCSimRoot installation inside the fiTQun image

Although the full WCSim executable is not available, the image does contain a complete WCSimRoot installation. This was checked with:

```
apptainer exec fiTQun_latest.sif bash fiTQun_env.sh \
    find /usr/local/hk/WCSimRoot -type f | head -100
```

The WCSimRoot installation contains headers, CMake configuration files, ROOT dictionaries and the shared library:

```
/usr/local/hk/WCSimRoot/install-Linux_x86_64-gcc_8-python_3.12.1/include/WCSimRoot/WCSimRootEvent.hh
/usr/local/hk/WCSimRoot/install-Linux_x86_64-gcc_8-python_3.12.1/include/WCSimRoot/WCSimRootGeom.hh
/usr/local/hk/WCSimRoot/install-Linux_x86_64-gcc_8-python_3.12.1/include/WCSimRoot/WCSimRootOptions.hh
/usr/local/hk/WCSimRoot/install-Linux_x86_64-gcc_8-python_3.12.1/lib/cmake/WCSimRoot/WCSimRootConfig.
    cmake
/usr/local/hk/WCSimRoot/install-Linux_x86_64-gcc_8-python_3.12.1/lib/libWCSimRoot.so.1.12.29
/usr/local/hk/WCSimRoot/install-Linux_x86_64-gcc_8-python_3.12.1/lib/libWCSimRoot_rdict.pcm
```

The installed WCSimRoot version used by fiTQun is therefore:

```
WCSimRoot 1.12.29
```

This confirms that the image is equipped to read WCSim ROOT files through the WCSimRoot classes, but not to run the full WCSim simulation.

3.6 Runtime library dependencies of runfiTQun

The runtime dependencies of `runfiTQun` were checked showing that `runfiTQun` is linked against ROOT libraries, `WCSimRoot` and `ToolFrameworkCore`. The most relevant lines are:

```
libCore.so => /usr/local/hk/ROOT/install-Linux_x86_64-gcc_8-python_3.12.1/lib/libCore.so
libRIO.so => /usr/local/hk/ROOT/install-Linux_x86_64-gcc_8-python_3.12.1/lib/libRIO.so
libTree.so => /usr/local/hk/ROOT/install-Linux_x86_64-gcc_8-python_3.12.1/lib/libTree.so
libWCSimRoot.so.1.12.29 => /usr/local/hk/WCSimRoot/install-Linux_x86_64-gcc_8-python_3.12.1/lib/
    libWCSimRoot.so.1.12.29
libToolFrameworkCore.so.3.0.0 => /usr/local/hk/ToolFrameworkCore/install-Linux_x86_64-gcc_8-python_3
    .12.1/lib/libToolFrameworkCore.so.3.0.0
```

No dependency on Geant4 libraries or on the full WCSim simulation library was found. In particular, there was no evidence of dependencies such as:

```
libG4run.so
libG4tracking.so
libG4geometry.so
libWCSimCore.so
```

This is the main technical evidence that the `fiTQun:latest` image is a reconstruction image, not a complete simulation image.

3.7 Compatibility with full WCSim and unique image fiTQun+WCSim

From a software-interface point of view, fiTQun is compatible with WCSim through WCSimRoot. In other words, fiTQun does not need to be linked directly against the full WCSim simulation executable. It only needs a WCSimRoot version that is compatible with the ROOT files produced by WCSim.

For this reason, the most important compatibility point is the WCSimRoot version. The existing image uses:

```
WCSimRoot 1.12.29
```

If a full WCSim installation produces ROOT files using a different WCSimRoot schema, fiTQun may still run, but compatibility is not guaranteed.
To obtain a single image capable of both generating WCSim events and reconstructing them with fiTQun, two possible strategies exist:

- start from the existing `fiTQun_latest.sif` image and add Geant4, the Geant4 data files and a full WCSim build;
- start from a full WCSim image and build fiTQun on top of the same ROOT and WCSimRoot installation.

The second strategy is cleaner for production use. WCSim is the more constrained package, because it depends on Geant4, the Geant4 data files, ROOT and the compiler ABI. Once a consistent WCSim installation exists, fiTQun can be compiled against its WCSimRoot installation. This reduces the risk of having two different WCSimRoot versions in the same image.

4 Building of fiTQun+WCSim image

4.1 Short summary

The starting point of this work was an already functional `fiTQun` Apptainer image:

```
fiTQun_latest.sif
```

This image contained:

- `fiTQun`;
- `ROOT 6.26/16`;
- `WCSimRoot 1.12.29`;

but did not contain:

- the full WCSim executable;
- `Geant4`;

- the Geant4 data files.

To obtain a single image capable of both simulating and reconstructing events, a new Apptainer image was built starting from `fiTQun_latest.sif`. The new image added:

- `Geant4 10.3.3`;
- the required Geant4 data files;
- a full WCSim build;
- the WCSim executable and macros.

The resulting image was:

```
fiTQun_WCSim.sif
```

The environment script inside the new image was:

```
fiTQun_WCSim_env.sh
```

This shell script is generated outside the container and it is built as executable with

```
chmod +x fiTQun_WCSim_env.sh
```

and the basic checks confirmed that the relevant executables were available:

```
apptainer exec ./fiTQun_WCSim.sif \
    bash fiTQun_WCSim_env.sh which WCSim

apptainer exec ./fiTQun_WCSim.sif \
    bash fiTQun_WCSim_env.sh which geant4-config

apptainer exec ./fiTQun_WCSim.sif \
    bash fiTQun_WCSim_env.sh which runfiTQun

apptainer exec ./fiTQun_WCSim.sif \
    bash fiTQun_WCSim_env.sh which root
```

The output showed:

```
/usr/local/hk/WCSimFull/install/bin/WCSim
/usr/local/hk/Geant4/install-10.3.3/bin/geant4-config
/usr/local/hk/fiTQun/install-Linux_x86_64-gcc_8-python_3.12.1/bin/runfiTQun
/usr/local/hk/ROOT/install-Linux_x86_64-gcc_8-python_3.12.1/bin/root
```

The Geant4 data files were also correctly available, as shown by environment variables such as:

```
G4LEDATA
G4NEUTRONHPDATA
G4LEVELGAMMADATA
G4RADIOACTIVEDATA
G4ENSDFSTATEDATA
```

Finally, running:

```
apptainer exec ./fiTQun_WCSim.sif \
    bash fiTQun_WCSim_env.sh WCSim --help
```

confirmed that WCSim could start correctly and detect the Geant4 installation.

4.2 Building the combined fiTQun + WCSim image with a docker file

The build was performed using an Apptainer definition file:

```
fiTQun_WCSim.def
```

The complete script is shown in Appendix B. The image is built with:

```
apptainer build fiTQun_WCSim.sif fiTQun_WCSim.def
```

or, if root privileges are required:

```
sudo apptainer build fiTQun_WCSim.sif fiTQun_WCSim.def
```

The definition file uses the local fiTQun image as its starting point:

```
Bootstrap: localimage
From: ./fiTQun_latest.sif
```

This means that the new image does not rebuild fiTQun from source. Instead, it inherits the already working fiTQun installation from `fiTQun_latest.sif` and adds the missing simulation components.

4.2.1 Starting point: the existing fiTQun image

The first part of the build script defines the paths already present in the original fiTQun image:

```
export FITQUN_SRC=/usr/local/hk/fiTQun
export FITQUN_INSTALL=/usr/local/hk/fiTQun/install-Linux_x86_64-gcc_8-python_3.12.1
export WCSIMROOT_INSTALL=/usr/local/hk/WCSimRoot/install-Linux_x86_64-gcc_8-python_3.12.1
export ROOT_INSTALL=/usr/local/hk/ROOT/install-Linux_x86_64-gcc_8-python_3.12.1
```

These paths correspond to:

- the fiTQun source directory;
- the fiTQun installation directory;
- the WCSimRoot installation already used by fiTQun;
- the ROOT installation inherited from the original image.

The corresponding runtime variables are also set:

```
export FITQUN_ROOT=${FITQUN_SRC}

export PATH=${FITQUN_INSTALL}/bin:${ROOT_INSTALL}/bin:$PATH
export LD_LIBRARY_PATH=${FITQUN_INSTALL}/lib:${WCSIMROOT_INSTALL}/lib:${ROOT_INSTALL}/lib:
    $LD_LIBRARY_PATH
export ROOT_INCLUDE_PATH=${WCSIMROOT_INSTALL}/include/WCSimRoot:${WCSIMROOT_INSTALL}/include:
    $ROOT_INCLUDE_PATH
```

This is needed so that, during the build, the already installed ROOT and fiTQun components can be found. In particular, ROOT is initialized with:

```
if [ -f ${ROOT_INSTALL}/bin/thisroot.sh ]; then
    source ${ROOT_INSTALL}/bin/thisroot.sh
fi
```

and its version is checked with:

```
root-config --version
```

4.2.2 Installing build tools

The image then installs the tools needed to compile Geant4 and WCSim:

```
apt-get update
apt-get install -y git cmake make gcc g++ wget curl ca-certificates expat libexpat1-dev
apt-get clean
rm -rf /var/lib/apt/lists/*
```

Alternative commands are included for systems using `dnf` or `yum`:

```
dnf install -y git cmake make gcc gcc-c++ wget curl ca-certificates expat-devel
```

or:

```
yum install -y git cmake make gcc gcc-c++ wget curl ca-certificates expat-devel
```

These packages are needed because the image has to download, configure and compile new software during the `%post` stage. In particular:

- `git` is needed to clone WCSim;
- `cmake` is used to configure both Geant4 and WCSim;
- `gcc` and `g++` are the compilers;
- `make` performs the compilation;
- `wget` is used to download the Geant4 source archive;
- `expat` and `libexpat1-dev` are required by Geant4.

4.2.3 Installing Geant4

The next step installs Geant4:

```
export GEANT4_VERSION=10.3.3
export GEANT4_HOME=/usr/local/hk/Geant4
export GEANT4_SRC=${GEANT4_HOME}/geant4.${GEANT4_VERSION}
export GEANT4_BUILD=${GEANT4_HOME}/build-${GEANT4_VERSION}
export GEANT4_INSTALL=${GEANT4_HOME}/install-${GEANT4_VERSION}
```

Version `10.3.3` was chosen because it is the Geant4 version indicated as a supported reference version for WCSim in the Hyper-K software container environment. The source code is downloaded from the Geant4 GitLab repository:

```
wget -O geant4.${GEANT4_VERSION}.tar.gz \
    https://gitlab.cern.ch/geant4/geant4/-/archive/v${GEANT4_VERSION}/geant4-v${GEANT4_VERSION}.tar.gz

tar -xzf geant4.${GEANT4_VERSION}.tar.gz
mv geant4-v${GEANT4_VERSION} ${GEANT4_SRC}
```

Then a separate build directory is created:

```
mkdir -p ${GEANT4_BUILD}
cd ${GEANT4_BUILD}
```

Geant4 is configured with CMake:

```
cmake ${GEANT4_SRC} \
    -DCMAKE_INSTALL_PREFIX=${GEANT4_INSTALL} \
    -DGEANT4_INSTALL_DATA=ON \
    -DGEANT4_USE_OPENGL_X11=OFF \
    -DGEANT4_USE_QT=OFF \
    -DGEANT4_BUILD_MULTITHREADED=ON
```

The most important option is:

```
-DGEANT4_INSTALL_DATA=ON
```

because WCSim needs the Geant4 external data files at runtime. Without these datasets, WCSim can fail with errors such as:

```
G4ENSDFSTATEDATA environment variable must be set
```

Graphical options are disabled:

```
-DGEANT4_USE_OPENGL_X11=OFF
-DGEANT4_USE_QT=OFF
```

because the image is intended mainly for batch simulation and reconstruction, not for interactive graphical visualization. Multithreading is enabled with:

```
-DGEANT4_BUILD_MULTITHREADED=ON
```

Finally, Geant4 is compiled and installed:

```
make -j$(nproc)
make install
```

and its environment is initialized:

```
source ${GEANT4_INSTALL}/bin/geant4.sh
```

The installed version is checked with:

```
geant4-config --version
```

4.2.4 Installing full WCSim

After Geant4 has been installed, the full WCSim package is built. The relevant paths are:

```
export WCSIM_HOME=/usr/local/hk/WCSimFull
export WCSIM_SOURCE_DIR=${WCSIM_HOME}/WCSim
export WCSIM_BUILD_DIR=${WCSIM_HOME}/build
export WCSIM_INSTALL_DIR=${WCSIM_HOME}/install
```

The WCSim source code is cloned from GitHub:

```
git clone https://github.com/WCSim/WCSim.git ${WCSIM_SOURCE_DIR}
```

A separate build directory is created:

```
mkdir -p ${WCSIM_BUILD_DIR}
cd ${WCSIM_BUILD_DIR}
```

WCSim is configured with:

```
cmake ${WCSIM_SOURCE_DIR} \
    -DCMAKE_INSTALL_PREFIX=${WCSIM_INSTALL_DIR} \
    -DWCSim_WCSimRoot_only=OFF
```

The key option is:

```
-DWCSim_WCSimRoot_only=OFF
```

This explicitly requests a full WCSim build, not only the WCSimRoot library. This is essential because the original fiTQun image already contained WCSimRoot, but did not contain the full WCSim executable. With this option, the build produces the actual simulation program:

```
WCSim
```

WCSim is then compiled and installed:

```
make -j$(nproc)
make install
```

At this point, the image contains:

- the original fiTQun installation inherited from `fiTQun_latest.sif`;
- the original ROOT installation inherited from `fiTQun_latest.sif`;
- the original WCSimRoot installation used by fiTQun;
- a new Geant4 installation;
- a new full WCSim installation.

IMPORTANT NOTE The full WCSim installation added to the combined image was not obtained by reproducing the complete WCSim GitHub Actions validation workflow. Instead, a manual CMake-based installation was performed. The procedure follows the standard WCSim source-build approach: clone the WCSim repository, configure it with CMake, compile it and install it. The option `WCSim_WCSimRoot_only=OFF` was used to force a full WCSim build rather than a WCSimRoot-only installation.
This differs from the official WCSim CI workflow, which uses a dedicated Hyper-K software base image and runs the WCSim validation chain, as described in section 2.4.

4.2.5 Environment script written inside the image

The definition file also writes an environment script inside the image:

```
/usr/local/hk/fitqun_wcsim_env.sh
```

However, this internal script is redundant with respect to the external script shown in Appendix A:

```
fiTQun_WCSim_env.sh
```

which is the one actually used in this work. The external script is preferred because it is more complete and can be modified without rebuilding the image. Therefore, all validation commands were run using:

```
apptainer exec ./fiTQun_WCSim.sif bash fiTQun_WCSim_env.sh <command>
```

rather than the internal script.

4.2.6 Version report

The build also writes a version report:

```
/usr/local/hk/versions_fitqun_wcsim.txt
```

The file records:

- the build date;
- the ROOT version;
- the Geant4 version;
- the WCSim commit;
- the fiTQun installation path;
- the original WCSimRoot installation path;
- the full WCSim installation path.

The WCSim commit is stored with:

```
git rev-parse HEAD
```

This is important for reproducibility, because the WCSim build uses the current state of the GitHub repository at build time.

4.3 Reminder: compatibility note

At this stage, the image contains two WCSimRoot installations:

- `WCSimRoot 1.12.29`, used by the pre-existing fiTQun build;
- `WCSimRoot 1.12.30`, produced by the full WCSim installation.

This was verified with:

```
ldd $(which runfiTQun) | grep -i WCSimRoot
ldd $(which WCSim) | grep -i WCSimRoot
```

which showed that `runfiTQun` and `WCSim` are linked against different WCSimRoot versions.

4.4 Current status

The current test image successfully provides:

- a working WCSim executable;
- a working Geant4 installation;
- the required Geant4 data files;
- the original working fiTQun installation;
- WCSim macros, including Hyper-K related macros such as `WCSim_1part_HK.mac` and `tuning_parameters_hkfd.mac`.

The next validation step is to generate a small WCSim ROOT file, for example with one event, and then check whether the existing `runfiTQun` executable can read and reconstruct it. If this works, the image can already be used as a practical test environment. If it fails because of WCSimRoot schema or library compatibility issues, the next step will be to rebuild fiTQun against the WCSimRoot version installed together with WCSim.

5 Shell script `fiTQun_WCSim_env.sh`: useful path

5.1 Runtime environment script

An environment script was needed in order to make all executables, libraries and data files visible at runtime. The script used in this work, as shown in Appendix A, is:

```
fiTQun_WCSim_env.sh
```

This is necessary because the different packages are installed in separate directories under:

```
/usr/local/hk
```

and the shell does not automatically know where to find their executables, libraries, CMake files, headers and Geant4 datasets.

5.1.1 Main package locations

The first part of the script defines the main installation directories:

```
export HK_ROOT=/usr/local/hk

export FITQUN_SRC=${HK_ROOT}/fiTQun
export FITQUN_INSTALL=${HK_ROOT}/fiTQun/install-Linux_x86_64-gcc_8-python_3.12.1

export ROOT_INSTALL=${HK_ROOT}/ROOT/install-Linux_x86_64-gcc_8-python_3.12.1
export WCSIMROOT_INSTALL=${HK_ROOT}/WCSimRoot/install-Linux_x86_64-gcc_8-python_3.12.1
export TOOLFRAMEWORKCORE_INSTALL=${HK_ROOT}/ToolFrameworkCore/install-Linux_x86_64-gcc_8-python_3.12.1

export WCSIMFULL_INSTALL=${HK_ROOT}/WCSimFull/install
export GEANT4_INSTALL=${HK_ROOT}/Geant4/install-10.3.3
```

These variables are shortcuts to the relevant packages installed inside the image. In particular:

- `FITQUN_INSTALL` points to the installed fiTQun executable and libraries;
- `ROOT_INSTALL` points to the ROOT installation used by both WCSim and fiTQun;
- `WCSIMROOT_INSTALL` points to the WCSimRoot installation used by the original fiTQun build;
- `TOOLFRAMEWORKCORE_INSTALL` points to the ToolFrameworkCore libraries required by fiTQun;
- `WCSIMFULL_INSTALL` points to the full WCSim installation, including the `WCSim` executable;
- `GEANT4_INSTALL` points to the Geant4 installation used by WCSim.

This separation is useful because fiTQun and WCSim do not have exactly the same runtime requirements. fiTQun mainly needs ROOT, WCSimRoot and ToolFrameworkCore, whereas WCSim also needs Geant4 and the Geant4 data files.

5.1.2 fiTQun configuration path

The script then defines:

```
export FITQUN_ROOT=${FITQUN_SRC}
```

This variable is used by fiTQun to locate its configuration and parameter files. Without it, `runfiTQun` may not be able to find the files that define the reconstruction settings, constants and tuning inputs. Therefore, this variable is needed even if the `runfiTQun` executable itself is already visible in `PATH`.

5.1.3 Geant4 data files

```
export G4DATA=${GEANT4_INSTALL}/share/Geant4-10.3.3/data

export G4LEDATA=${G4DATA}/G4EMLOW6.50
export G4LEVELGAMMADATA=${G4DATA}/PhotonEvaporation4.3.2
export G4NEUTRONHPDATA=${G4DATA}/G4NDL4.5
export G4RADIOACTIVEDATA=${G4DATA}/RadioactiveDecay5.1.1
export G4REALSURFACEDATA=${G4DATA}/RealSurface1.0
export G4SAIDXSDATA=${G4DATA}/G4SAIDDATA1.1
export G4ENSDFSTATEDATA=${G4DATA}/G4ENSDFSTATE2.1
```

These variables tell Geant4 where to find the external datasets used by its physics models. They are not optional for a full WCSim run. For example:

- `G4LEDATA` is used by electromagnetic low-energy processes;
- `G4NEUTRONHPDATA` is used by high-precision neutron models;
- `G4LEVELGAMMADATA` is used for photon evaporation data;
- `G4RADIOACTIVEDATA` is used for radioactive decay data;
- `G4REALSURFACEDATA` is used for optical surface properties;
- `G4SAIDXSDATA` is used for some hadronic cross-section data;
- `G4ENSDFSTATEDATA` is used by the Geant4 nuclide table.

This part of the script was required because WCSim initially failed with the error:

```
G4ENSDFSTATEDATA environment variable must be set
```

After exporting the Geant4 data variables, the command:

```
apptainer exec ./fiTQun_WCSim.sif bash fiTQun_WCSim_env.sh WCSim --help
```

successfully started WCSim and printed the Geant4 banner and the WCSim usage message. This confirmed that Geant4 could now locate the required data files.

5.1.4 Executable search path

The script modifies the executable search path with:

```
export PATH=${WCSIMFULL_INSTALL}/bin:${GEANT4_INSTALL}/bin:${FITQUN_INSTALL}/bin:${ROOT_INSTALL}/bin:${
    PATH:-}
```

The shell uses `PATH` to find executables when a command is typed. In this case, the following commands must be found:

- `WCSim`, located in `WCSIMFULL_INSTALL/bin`;
- `geant4-config`, located in `GEANT4_INSTALL/bin`;
- `runfiTQun`, located in `FITQUN_INSTALL/bin`;
- `root` and `root-config`, located in `ROOT_INSTALL/bin`.

Without this line, these programs may still exist inside the image, but commands such as:

```
which WCSim
which geant4-config
which runfiTQun
which root
```

would fail unless their full absolute paths were used.

5.1.5 Dynamic library search path

The next important variable is `LD_LIBRARY_PATH`:

```
export LD_LIBRARY_PATH=${WCSIMFULL_INSTALL}/lib:${WCSIMFULL_INSTALL}/lib64:${GEANT4_INSTALL}/lib64:${
    GEANT4_INSTALL}/lib:${FITQUN_INSTALL}/lib:${WCSIMROOT_INSTALL}/lib:${TOOLFRAMEWORKCORE_INSTALL}/lib:$
    {ROOT_INSTALL}/lib:/.singularity.d/libs:${LD_LIBRARY_PATH:-}
```

This variable tells the dynamic linker where to look for shared libraries at runtime. This is different from `PATH`: `PATH` finds executables, whereas `LD_LIBRARY_PATH` finds the libraries needed by those executables after they have been launched.
For example:

- `runfiTQun` needs libraries from fiTQun, WCSimRoot, ToolFrameworkCore and ROOT;
- `WCSim` needs libraries from WCSim, Geant4 and ROOT;
- ROOT itself needs its own shared libraries and dictionaries.

If this variable is incomplete, the executable may be found by `which`, but fail at runtime with an error such as:

```
error while loading shared libraries: libXYZ.so: cannot open shared object file
```

5.1.6 ROOT include path

The script also defines:

```
export ROOT_INCLUDE_PATH=${WCSIMROOT_INSTALL}/include/WCSimRoot:${WCSIMROOT_INSTALL}/include:${
    ROOT_INCLUDE_PATH:-}
```

This variable is used by ROOT when loading headers, dictionaries and interpreted code. It is especially useful when ROOT needs to understand classes from WCSimRoot, such as:

```
WCSimRootEvent
WCSimRootGeom
WCSimRootOptions
```

These classes describe the objects stored in WCSim ROOT files. Therefore, `ROOT_INCLUDE_PATH` helps ROOT locate the corresponding header files when reading or inspecting WCSim output.

5.1.7 Execution of the requested command

The script ends with:

```
exec "$@"
```

This means that the script first prepares the environment and then executes the command passed by the user. For example:

```
apptainer exec ./fiTQun_WCSim.sif bash fiTQun_WCSim_env.sh WCSim --help
```

first sets all the variables described above and then runs:

```
WCSim --help
```

This structure makes the script convenient for testing different commands without having to manually source the environment every time.

A Shell script `fiTQun_WCSim_env.sh`

Listing 1: Shell script for fiTQun+WCSim image path environment

```
#!/bin/bash

# ==============================
# fiTQun + WCSim container environment
# ==============================

# Stop only if an unset variable is used inside this setup script.
# Do not use "set -e" here, because this script is meant to execute arbitrary commands.
set -u

# ------------------------------
# Main package locations
# ------------------------------

export HK_ROOT=/usr/local/hk

export FITQUN_SRC=${HK_ROOT}/fiTQun
export FITQUN_INSTALL=${HK_ROOT}/fiTQun/install-Linux_x86_64-gcc_8-python_3.12.1

export ROOT_INSTALL=${HK_ROOT}/ROOT/install-Linux_x86_64-gcc_8-python_3.12.1
export WCSIMROOT_INSTALL=${HK_ROOT}/WCSimRoot/install-Linux_x86_64-gcc_8-python_3.12.1
export TOOLFRAMEWORKCORE_INSTALL=${HK_ROOT}/ToolFrameworkCore/install-Linux_x86_64-gcc_8
    -python_3.12.1

export WCSIMFULL_INSTALL=${HK_ROOT}/WCSimFull/install
export GEANT4_INSTALL=${HK_ROOT}/Geant4/install-10.3.3

# ------------------------------
# fiTQun variables
# ------------------------------

# fiTQun uses this to find parameter/configuration files.
export FITQUN_ROOT=${FITQUN_SRC}

# ------------------------------
# Geant4 data files
# ------------------------------

export G4DATA=${GEANT4_INSTALL}/share/Geant4-10.3.3/data

export G4LEDATA=${G4DATA}/G4EMLOW6.50
export G4LEVELGAMMADATA=${G4DATA}/PhotonEvaporation4.3.2
export G4NEUTRONHPDATA=${G4DATA}/G4NDL4.5
export G4RADIOACTIVEDATA=${G4DATA}/RadioactiveDecay5.1.1
export G4REALSURFACEDATA=${G4DATA}/RealSurface1.0
export G4SAIDXSDATA=${G4DATA}/G4SAIDDATA1.1
export G4ENSDFSTATEDATA=${G4DATA}/G4ENSDFSTATE2.1

# ------------------------------
# Executables
# ------------------------------

export PATH=${WCSIMFULL_INSTALL}/bin:${GEANT4_INSTALL}/bin:${FITQUN_INSTALL}/bin:${
    ROOT_INSTALL}/bin:${PATH:-}

# ------------------------------
# Dynamic libraries
# ------------------------------

```

```
export LD_LIBRARY_PATH=${WCSIMFULL_INSTALL}/lib:${WCSIMFULL_INSTALL}/lib64:${
    GEANT4_INSTALL}/lib64:${GEANT4_INSTALL}/lib:${FITQUN_INSTALL}/lib:${
    WCSIMROOT_INSTALL}/lib:${TOOLFRAMEWORKCORE_INSTALL}/lib:${ROOT_INSTALL}/lib:/.
    singularity.d/libs:${LD_LIBRARY_PATH:-}

# ------------------------------
# ROOT include path
# ------------------------------

export ROOT_INCLUDE_PATH=${WCSIMROOT_INSTALL}/include/WCSimRoot:${WCSIMROOT_INSTALL}/
    include:${ROOT_INCLUDE_PATH:-}

# ------------------------------
# Run the command passed to this script
# ------------------------------

exec "$@"
```

B Docker file to build fiTQun+WCSim image

Listing 2: Docker file

```
Bootstrap: localimage
From: ./fiTQun_latest.sif

%post
    set -e

    echo "=== Building test image: fiTQun + Geant4 + full WCSim ==="

    # ============================================================
    # 1. Path gi  presenti nell'immagine fiTQun
    # ============================================================

    export FITQUN_SRC=/usr/local/hk/fiTQun
    export FITQUN_INSTALL=/usr/local/hk/fiTQun/install-Linux_x86_64-gcc_8-python_3.12.1
    export WCSIMROOT_INSTALL=/usr/local/hk/WCSimRoot/install-Linux_x86_64-gcc_8-python_3
        .12.1
    export ROOT_INSTALL=/usr/local/hk/ROOT/install-Linux_x86_64-gcc_8-python_3.12.1

    export FITQUN_ROOT=${FITQUN_SRC}

    export PATH=${FITQUN_INSTALL}/bin:${ROOT_INSTALL}/bin:$PATH
    export LD_LIBRARY_PATH=${FITQUN_INSTALL}/lib:${WCSIMROOT_INSTALL}/lib:${ROOT_INSTALL
        }/lib:$LD_LIBRARY_PATH
    export ROOT_INCLUDE_PATH=${WCSIMROOT_INSTALL}/include/WCSimRoot:${WCSIMROOT_INSTALL
        }/include:$ROOT_INCLUDE_PATH

    # ROOT setup
    if [ -f ${ROOT_INSTALL}/bin/thisroot.sh ]; then
        source ${ROOT_INSTALL}/bin/thisroot.sh
    fi

    echo "ROOT version:"
    root-config --version

    # ============================================================
    # 2. Installa strumenti di compilazione se mancano
    # ============================================================

    if command -v apt-get >/dev/null 2>&1; then
        apt-get update
```

```
        apt-get install -y git cmake make gcc g++ wget curl ca-certificates expat
            libexpat1-dev
        apt-get clean
        rm -rf /var/lib/apt/lists/*
    elif command -v dnf >/dev/null 2>&1; then
        dnf install -y git cmake make gcc gcc-c++ wget curl ca-certificates expat-devel
        dnf clean all
    elif command -v yum >/dev/null 2>&1; then
        yum install -y git cmake make gcc gcc-c++ wget curl ca-certificates expat-devel
        yum clean all
    else
        echo "ERROR: no apt-get/dnf/yum found. Install build tools manually or use
            another base image."
        exit 1
    fi

    # ============================================================
    # 3. Installa Geant4
    # ============================================================
    #
    # Nota: WCSim README indica Geant4 10.3.3 come versione supportata
    # nel container hk-software. Qui usiamo quella per ridurre il rischio.
    # ============================================================

    export GEANT4_VERSION=10.3.3
    export GEANT4_HOME=/usr/local/hk/Geant4
    export GEANT4_SRC=${GEANT4_HOME}/geant4.${GEANT4_VERSION}
    export GEANT4_BUILD=${GEANT4_HOME}/build-${GEANT4_VERSION}
    export GEANT4_INSTALL=${GEANT4_HOME}/install-${GEANT4_VERSION}

    mkdir -p ${GEANT4_HOME}
    cd ${GEANT4_HOME}

    wget -O geant4.${GEANT4_VERSION}.tar.gz https://gitlab.cern.ch/geant4/geant4/-/
        archive/v${GEANT4_VERSION}/geant4-v${GEANT4_VERSION}.tar.gz
    tar -xzf geant4.${GEANT4_VERSION}.tar.gz
    mv geant4-v${GEANT4_VERSION} ${GEANT4_SRC}

    mkdir -p ${GEANT4_BUILD}
    cd ${GEANT4_BUILD}

    cmake ${GEANT4_SRC} \
        -DCMAKE_INSTALL_PREFIX=${GEANT4_INSTALL} \
        -DGEANT4_INSTALL_DATA=ON \
        -DGEANT4_USE_OPENGL_X11=OFF \
        -DGEANT4_USE_QT=OFF \
        -DGEANT4_BUILD_MULTITHREADED=ON

    make -j$(nproc)
    make install

    source ${GEANT4_INSTALL}/bin/geant4.sh

    echo "Geant4 version:"
    geant4-config --version

    # ============================================================
    # 4. Installa WCSim completo
    # ============================================================

    export WCSIM_HOME=/usr/local/hk/WCSimFull
    export WCSIM_SOURCE_DIR=${WCSIM_HOME}/WCSim
    export WCSIM_BUILD_DIR=${WCSIM_HOME}/build
    export WCSIM_INSTALL_DIR=${WCSIM_HOME}/install
```

```
    mkdir -p ${WCSIM_HOME}

    git clone https://github.com/WCSim/WCSim.git ${WCSIM_SOURCE_DIR}

    cd ${WCSIM_SOURCE_DIR}

    mkdir -p ${WCSIM_BUILD_DIR}
    cd ${WCSIM_BUILD_DIR}

    cmake ${WCSIM_SOURCE_DIR} \
        -DCMAKE_INSTALL_PREFIX=${WCSIM_INSTALL_DIR} \
        -DWCSim_WCSimRoot_only=OFF

    make -j$(nproc)
    make install

    # ============================================================
    # 5. Script ambiente finale
    # ============================================================

    cat > /usr/local/hk/fitqun_wcsim_env.sh <<'EOF'
#!/bin/bash

export FITQUN_SRC=/usr/local/hk/fiTQun
export FITQUN_INSTALL=/usr/local/hk/fiTQun/install-Linux_x86_64-gcc_8-python_3.12.1
export OLD_WCSIMROOT_INSTALL=/usr/local/hk/WCSimRoot/install-Linux_x86_64-gcc_8-python_3.12.1
export ROOT_INSTALL=/usr/local/hk/ROOT/install-Linux_x86_64-gcc_8-python_3.12.1

export GEANT4_INSTALL=/usr/local/hk/Geant4/install-10.3.3

export WCSIM_HOME=/usr/local/hk/WCSimFull
export WCSIM_SOURCE_DIR=/usr/local/hk/WCSimFull/WCSim
export WCSIM_BUILD_DIR=/usr/local/hk/WCSimFull/build
export WCSIM_INSTALL_DIR=/usr/local/hk/WCSimFull/install

export FITQUN_ROOT=$FITQUN_SRC

if [ -f ${ROOT_INSTALL}/bin/thisroot.sh ]; then
    source ${ROOT_INSTALL}/bin/thisroot.sh
fi

if [ -f ${GEANT4_INSTALL}/bin/geant4.sh ]; then
    source ${GEANT4_INSTALL}/bin/geant4.sh
fi

if [ -f ${WCSIM_INSTALL_DIR}/bin/this_wcsim.sh ]; then
    source ${WCSIM_INSTALL_DIR}/bin/this_wcsim.sh
fi

export PATH=$FITQUN_INSTALL/bin:$WCSIM_INSTALL_DIR/bin:$ROOT_INSTALL/bin:$PATH

# Nota:
# per fiTQun si tiene il vecchio WCSimRoot gi  funzionante.
# per WCSim usiamo anche le librerie della nuova installazione.
export LD_LIBRARY_PATH=$FITQUN_INSTALL/lib:$OLD_WCSIMROOT_INSTALL/lib:$WCSIM_INSTALL_DIR/lib:$ROOT_INSTALL/lib:/.singularity.d/libs:$LD_LIBRARY_PATH

export ROOT_INCLUDE_PATH=$OLD_WCSIMROOT_INSTALL/include/WCSimRoot:$OLD_WCSIMROOT_INSTALL/include:$WCSIM_INSTALL_DIR/include:$ROOT_INCLUDE_PATH

exec "$@"
EOF
```

```
    chmod +x /usr/local/hk/fitqun_wcsim_env.sh

    # Version report
    cat > /usr/local/hk/versions_fitqun_wcsim.txt <<EOF
Build date: $(date)

ROOT:
$(root-config --version)

Geant4:
$(geant4-config --version)

WCSim commit:
$(cd ${WCSIM_SOURCE_DIR} && git rev-parse HEAD)

fiTQun install:
${FITQUN_INSTALL}

Old WCSimRoot install:
${WCSIMROOT_INSTALL}

WCSim full install:
${WCSIM_INSTALL_DIR}
EOF

%environment
    export FITQUN_SRC=/usr/local/hk/fiTQun
    export FITQUN_INSTALL=/usr/local/hk/fiTQun/install-Linux_x86_64-gcc_8-python_3.12.1
    export ROOT_INSTALL=/usr/local/hk/ROOT/install-Linux_x86_64-gcc_8-python_3.12.1
    export GEANT4_INSTALL=/usr/local/hk/Geant4/install-10.3.3
    export WCSIM_HOME=/usr/local/hk/WCSimFull
    export WCSIM_SOURCE_DIR=/usr/local/hk/WCSimFull/WCSim
    export WCSIM_BUILD_DIR=/usr/local/hk/WCSimFull/build
    export WCSIM_INSTALL_DIR=/usr/local/hk/WCSimFull/install

%runscript
    echo "fiTQun + full WCSim test image"
    echo "Use:"
    echo "  bash /usr/local/hk/fitqun_wcsim_env.sh which WCSim"
    echo "  bash /usr/local/hk/fitqun_wcsim_env.sh which runfiTQun"
```